

# Real Estate Development Intelligence Platform Engineering Master Plan

---

|                                                                              |
|------------------------------------------------------------------------------|
| Real Estate Development Intelligence Platform                                |
| Development-Ready Technical Master Plan                                      |
| Phase 1 — Source → Implementation Matrix (condensed)                         |
| Phase 2 — Architecture Validation                                            |
| Phase 3 — Technology Stack (final decisions)                                 |
| Phase 4 — Repository Structure                                               |
| Phase 5 — Domain Model (delta from Blueprint)                                |
| Phase 6 — PostGIS Engineering                                                |
| Phase 7 — GeoJSON Contract                                                   |
| Phase 8 — Computational Geometry Engine                                      |
| Phase 9 — Geometry Library Decision (verified)                               |
| Phase 10 — Constraint Engine                                                 |
| Phase 11 — Regulation Engine: Rule DSL and Deterministic Evaluator           |
| Phase 12 — Regulation RAG (ingestion pipeline)                               |
| Phase 13 — Feasibility Engine as an Executable DAG                           |
| Phase 14 — Generative Layout Engine & Unit Packing                           |
| Phase 15 — Parking / Mobility Engine                                         |
| Phase 16 — Structural Grid (conceptual only)                                 |
| Phase 17 — 3D Massing Engine                                                 |
| Phase 18 — Financial Feasibility Engine                                      |
| Phase 19 — Scenario Engine                                                   |
| Phase 20 — Multi-Objective Optimization Engine (NSGA-II)                     |
| Phase 21 — AI Development Copilot                                            |
| Phase 22 — API Specification (OpenAPI 3.1, key contracts)                    |
| Phase 23 — Worker Architecture                                               |
| Phase 24 — Frontend Architecture                                             |
| Phase 25 — CAD-Grade Map Editor                                              |
| Phase 26 — Versioning                                                        |
| Phase 27 — Testing                                                           |
| Phase 28 — Security                                                          |
| Phase 29 — Observability                                                     |
| Phase 30 — Deployment                                                        |
| Phase 31 — Cost Model                                                        |
| Phase 32 — Risk Register                                                     |
| Phase 33 — MVP (P0) Definition                                               |
| Phase 34 — 90-Day Engineering Roadmap                                        |
| Phase 35 — 6-12 Month Expansion Roadmap                                      |
| Phase 36 — Exact Implementation Order (single sequence, dependency-resolved) |
| Summary of Where This Plan Diverges From the Blueprint                       |

# Real Estate Development Intelligence Platform

---

## Development-Ready Technical Master Plan

---

**Source documents:** *Land Feasibility Platform Blueprint* and *Real Estate Feasibility Platform Blueprint* (two near-identical Planwise reverse-engineering deliverables, referred to jointly below as “the Blueprint”). The Planwise Site Development Masterclass PDF was not machine-readable in this session (image/slide-style PDF with no extractable text layer) and is therefore treated as **[UNKNOWN]** source content except where the two Blueprint documents already summarize it.

**Epistemic tags used throughout:** **[DOCUMENTED]** = stated in the Blueprint · **[INFERRED]** = technically implied · **[PROPOSED]** = new recommendation made here · **[EXTERNALLY VERIFIED]** = checked against library/standard behavior · **[UNKNOWN]** = not established by any source, not to be treated as fact.

This document does not restate the Blueprint’s 38-row geometry matrix, full statutory FSI tables, or full financial formula derivations verbatim — those are already correct and implementation-ready in the source PDFs and are referenced by name. What follows is the **engineering layer the Blueprint does not fully specify**: validated architecture decisions, the rule-evaluator internals, the DAG execution engine, worker service contracts, repository rationale, completed API surface, concrete test/deploy mechanics, and a buildable sequencing plan — i.e., “what code do I write.”

---

## Phase 1 — Source → Implementation Matrix (condensed)

---

| #  | Subsystem             | Blueprint Status                                                                        | Engineering Decision                                                                                                    |
|----|-----------------------|-----------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------|
| 1  | Auth                  | [DOCUMENTED] prototype used localStorage JWT (insecure)                                 | Replace with OAuth2/OIDC + short-lived access token in memory + HttpOnly refresh cookie. [PROPOSED]                     |
| 2  | Organizations         | [DOCUMENTED] multi-tenant model named, RBAC unspecified                                 | Postgres RLS + role enum on <code>users</code> . [PROPOSED]                                                             |
| 3  | Projects              | [DOCUMENTED] fields; state machine unspecified                                          | Explicit <code>project_status_type</code> enum + audit log on transition. [INFERRED]                                    |
| 4  | Sites/Parcels         | [DOCUMENTED] schema given in both blueprints, consistent                                | Adopt as-is with minor generated-column fixes (below). [DOCUMENTED]                                                     |
| 5  | GIS/GeoJSON           | [DOCUMENTED] WGS84 capture, RFC 7946                                                    | Add canonical FeatureCollection contract (Blueprint §6/§7) as Zod + Pydantic pair. [DOCUMENTED]                         |
| 6  | CAD drawing           | [DOCUMENTED] Mapbox Draw + snapping described; engine unbuilt                           | Custom draw-mode suite on MapLibre; Flatbush R-tree client index. [DOCUMENTED]                                          |
| 7  | Constraints           | [DOCUMENTED] schema and severities given                                                | Adopt; add <code>effective_from/effective_to</code> for conditional constraints (Phase 10 below). [INFERRED]            |
| 8  | Regulation            | [DOCUMENTED] JSON rule schema given; <b>evaluator engine not specified</b>              | Design deterministic AST compiler (Phase 11 below). [PROPOSED] — this is the single largest gap in the source material. |
| 9  | Geometry              | [DOCUMENTED] 32-38 operations, library mapping given, [EXTERNALLY VERIFIED] in this doc | Adopt library mapping; add service contract + numeric-precision policy (Phase 8 below).                                 |
| 10 | Feasibility           | [DOCUMENTED] pipeline sequence given as prose/math                                      | Formalize as executable DAG with node contracts (Phase 12 below). [PROPOSED]                                            |
| 11 | Layout generation     | [UNKNOWN] in original prototype; [PROPOSED] MILP in Blueprint                           | Adopt MILP for unit-demising, heuristic bay-grid for cores (Phase 13). [PROPOSED]                                       |
| 12 | Unit packing          | [PROPOSED] MILP, OR-Tools named                                                         | Confirmed correct choice; formulation given below. [EXTERNALLY VERIFIED]                                                |
| 13 | Parking               | [DOCUMENTED] stall/aisle/ramp math; layout algorithm not specified                      | Grid-packing + constraint solver, Reeds-Shepp validation as proposed by Blueprint. [PROPOSED]                           |
| 14 | Structural grid       | [DOCUMENTED] conceptual-only, span ranges given                                         | Adopt as non-engineering advisory layer only. [DOCUMENTED]                                                              |
| 15 | 3D massing            | [DOCUMENTED] Three.js custom-layer code given and verified                              | Adopt verbatim; it is correct, current API usage. [EXTERNALLY VERIFIED]                                                 |
| 16 | Financial             | [DOCUMENTED] full DCF formula set                                                       | Adopt; wrap in versioned pure-function engine (Phase 18). [DOCUMENTED]                                                  |
| 17 | Optimization          | [DOCUMENTED] NSGA-II selection, objectives named; chromosome encoding not specified     | Encoding defined below (Phase 21). [PROPOSED]                                                                           |
| 18 | AI copilot            | [DOCUMENTED] deterministic-gate pattern, correct                                        | Adopt; tool schemas defined below (Phase 22). [DOCUMENTED]                                                              |
| 19 | RAG                   | [DOCUMENTED] OCR→LayoutLM→Neo4j/Qdrant→human-verify pipeline                            | Adopt; add confidence-gate thresholds (Phase 12 in this doc). [DOCUMENTED]                                              |
| 20 | Document intelligence | [DOCUMENTED] LayoutLMv3 + Table-Transformer + Textract                                  | Adopt as separate worker pool, not inline with regulation engine. [INFERRED]                                            |
| 21 | Collaboration         | [DOCUMENTED] Yjs CRDT + Redis soft-locks                                                | Adopt; defer to Growth horizon, not MVP. [DOCUMENTED]                                                                   |
| 22 | Versioning            | [DOCUMENTED] SHA-256 input-digest audit concept                                         | Adopt; schema given (Phase 26). [DOCUMENTED]                                                                            |
| 23 | Reporting             | [DOCUMENTED] headless Chromium / Puppeteer                                              | Adopt as isolated worker pool with hard timeout. [DOCUMENTED]                                                           |
| 24 | Audit                 | [DOCUMENTED] append-only <code>calculation_audit_logs</code> table given                | Adopt; add row-level immutability trigger (Phase 26). [PROPOSED]                                                        |
| 25 | Telemetry             | [DOCUMENTED] OpenTelemetry/Prometheus/Grafana named, no instrumentation contract        | Add correlation-id propagation contract (Phase 29). [PROPOSED]                                                          |
| 26 | Security              | [DOCUMENTED] RLS, CSP, cookie auth all specified with working SQL/config                | Adopt as written; it is correct. [EXTERNALLY VERIFIED]                                                                  |
| 27 | Deployment            | [DOCUMENTED] AWS ECS Fargate + RDS + ElastiCache topology                               | Adopt for MVP/Growth; Kubernetes deferred (validated below). [DOCUMENTED]                                               |

**Unknowns that remain genuinely unresolved by either Blueprint and must not be treated as fact:** the original Planwise backend’s exact runtime/framework [UNKNOWN→INFERRED] ; whether any financial module (“FinWise”) shipped real formulas or only marketing copy [UNKNOWN] ; any existing integration with AutoDCR or other government submission portals [UNKNOWN] ; multi-tenant permission granularity in the original product [UNKNOWN] . Do not port fictitious behavior for these — the plan below designs them fresh.

## Phase 2 — Architecture Validation

---

The Blueprint selects **Architecture E: Hybrid Application + Workers**. Independent validation against the actual workload mix:

- Interactive workloads (project CRUD, auth, geometry auto-save round-trip) need **P95 < 300 ms**. A synchronous stateless API tier satisfies this; a full microservice mesh adds RPC latency with no corresponding benefit at this team size.
- Heavy workloads (straight-skeleton on complex polygons, MILP unit-packing, NSGA-II) run **seconds to minutes** and must never block an HTTP thread — this rules out plain monolith/modular-monolith for those code paths, confirming the need for a worker tier regardless of the web tier's shape.
- Team size implied by the 90-day plan (8–12 engineers) is below the threshold where microservice operational overhead (distributed tracing, service mesh, per-service on-call) pays for itself. Event-driven-everywhere (Kafka from day one) front-loads infrastructure the MVP doesn't need — message replay, exactly-once semantics, consumer-group rebalancing — for a workload where a single Redis-backed queue is sufficient until queue depth or multi-region requirements force it.
- Data consistency: statutory/financial figures are legally sensitive (Risk RSK-REG-02 in the Blueprint). A single ACID Postgres instance as source of truth, with workers writing results back transactionally, is safer than eventual-consistency sagas across services.

**Conclusion: Architecture E confirmed**, with one refinement — do not adopt Kafka at MVP even though the Blueprint's Tier 4 diagram shows it; use it only from the Enterprise horizon (Blueprint's own §3 "Architectural Evolution Strategy" already says this — Phase 3 text of the second Blueprint elsewhere shows Kafka in the day-one diagram, which is an internal inconsistency in the source; **the phased version is authoritative** and is adopted here).

### MVP Architecture (do not build anything beyond this list at MVP)

- One stateless API service (Go **or** FastAPI — decision below), one Postgres+PostGIS instance, one Redis instance (queue + cache, single logical store, can be split later), one geometry/regulation/financial worker pool (can start as one process type with multiple queues, split later), one report-render worker, S3-compatible object storage, Cloudflare in front.
- **Do NOT introduce at MVP:** Kubernetes, Kafka, Neo4j, Qdrant/vector DB, CRDT collaboration, multi-region, service mesh, separate microservice-per-domain. Each of these has a named, later trigger condition (below).

### Growth Architecture

- Split worker pool into Geometry / Generative-Design / Financial / Document-AI / Export pools (separate containers, separate autoscaling policies), add Postgres read replica, add Redis Cluster mode, introduce Qdrant + Neo4j for RAG once regulation-document ingestion volume justifies it (trigger: >3 jurisdictions or >50 documents ingested).
- Trigger for splitting the API service itself: sustained CPU saturation on the stateless tier unrelated to worker load, or org/team boundaries requiring independent deploy cadence.

### Enterprise Architecture

- Kafka for telemetry + document-scraping fan-out; KEDA-based autoscaling on queue depth; CRDT (Yjs) real-time collaboration; multi-region read replicas; dedicated vector-DB cluster. Trigger: >2,500 orgs or contractual multi-region/data-residency requirements (matches Blueprint's own cost-model growth stage).

---

## Phase 3 — Technology Stack (final decisions)

---

| Layer                | Choice                                                                                                                                                                                                                                                                                                                    | Why                                                                                                                                                                                                     | Key alternative rejected                                                                                                                                                                                                                  | Trigger to revisit                                                                                                                                   |
|----------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| Frontend framework   | <b>React 18/19 + TypeScript + Vite</b>                                                                                                                                                                                                                                                                                    | SPA with heavy canvas/WebGL state; Vite gives faster dev-loop than Next.js and MVP has no SSR/SEO need                                                                                                  | Next.js — adds SSR complexity the map canvas doesn't benefit from                                                                                                                                                                         | If public marketing pages need SSR, add Next.js as a <i>separate</i> app, not inside the workspace SPA                                               |
| Server state         | <b>TanStack Query v5</b>                                                                                                                                                                                                                                                                                                  | Exactly matches Blueprint's own recommendation; battle-tested cache/refetch for REST                                                                                                                    | Redux+RTK Query — more boilerplate for no material benefit here                                                                                                                                                                           | —                                                                                                                                                    |
| Client state         | <b>Zustand</b>                                                                                                                                                                                                                                                                                                            | Minimal API for ephemeral UI (tool mode, modal visibility); avoids Redux ceremony                                                                                                                       | Context API — re-render storms on high-frequency draw events                                                                                                                                                                              | —                                                                                                                                                    |
| Mapping              | <b>MapLibre GL JS v4</b>                                                                                                                                                                                                                                                                                                  | BSD-licensed, zero vendor lock-in, API-compatible with Mapbox GL JS v3 draw plugins                                                                                                                     | Mapbox GL JS — proprietary token/billing lock-in for a component with a free-to-use fork                                                                                                                                                  | If satellite basemap quality becomes a blocker, use Mapbox's tile <i>service</i> only, keep MapLibre as the renderer                                 |
| 3D                   | <b>Three.js r160+</b> as a custom MapLibre layer                                                                                                                                                                                                                                                                          | Verified working pattern in Blueprint \$7/\$15 with correct WebGL context sharing; lowest memory footprint of the four evaluated engines                                                                | CesiumJS — 120-250 MB base memory, wrong tool below city/region scale                                                                                                                                                                     | Regional (>100 km²) master planning → introduce Cesium as a <i>separate</i> macro-planning view                                                      |
| API/backend language | <b>Go</b> for the API gateway service                                                                                                                                                                                                                                                                                     | Static typing, single static binary, cheap concurrency for a mostly I/O-bound CRUD+auth tier, easy Fargate deployment                                                                                   | FastAPI — fine too, but Python's GIL and heavier runtime are a worse fit for the stateless tier when Python is <i>already</i> needed for the workers (keeps a clean separation of concerns: Go = transactional API, Python/C++ = compute) | If the team is Python-only, FastAPI is an acceptable substitute with no other architectural change                                                   |
| Worker language      | <b>Python 3.12</b> (geometry/regulation/financial/document-AI) with <b>C++ via CGAL bindings</b> for straight-skeleton and LIR (largest-inscribed-rectangle) only                                                                                                                                                         | Shapely/PyProj/GEOS bindings are mature; CGAL has no first-class Python bindings for straight skeleton, so that operation is isolated behind a small C++ microservice/CLI called via subprocess or gRPC | Pure-C++ workers — far slower iteration speed for regulation/financial logic that changes weekly                                                                                                                                          | —                                                                                                                                                    |
| Queue                | <b>Redis Streams + BullMQ (Node) or Celery (Python)</b> — use <b>Celery</b> since workers are Python                                                                                                                                                                                                                      | Simplicity at MVP scale; avoids RabbitMQ/Kafka operational cost                                                                                                                                         | RabbitMQ — better delivery guarantees but unnecessary operational surface at MVP                                                                                                                                                          | Sustained queue depth >10k or need for consumer-group semantics → introduce Kafka (Enterprise horizon only)                                          |
| Database             | <b>PostgreSQL 16 + PostGIS 3.4</b>                                                                                                                                                                                                                                                                                        | Only production-grade option that gives ACID + native spatial types + generated columns in one engine; Blueprint's schema already assumes it                                                            | —no real alternative for this workload—                                                                                                                                                                                                   | —                                                                                                                                                    |
| Geometry libs        | <b>GEOS/Shapely 2.0</b> (boolean ops, buffers, validity) · <b>Clipper2</b> (high-volume polygon clipping/offsetting where float precision tolerance is acceptable) · <b>CGAL</b> (straight skeleton, LIR, medial axis — exact-predicate operations only) · <b>Turf.js</b> (client-side preview only, never authoritative) | Matches Blueprint's Operation→Engine matrix; independently verified against each library's documented capability set                                                                                    | —                                                                                                                                                                                                                                         | —                                                                                                                                                    |
| Optimization         | <b>Google OR-Tools (CBC/SCIP)</b> for MILP unit-packing and podium/tower alignment · custom <b>NSGA-II</b> (via <code>pymoo</code> , not hand-rolled) for scenario optimization                                                                                                                                           | <code>pymoo</code> is a maintained, correctly-implemented NSGA-II with constraint-handling support — hand-rolling a genetic algorithm is a common source of subtle correctness bugs                     | DEAP — lower-level, more code to maintain for equivalent result                                                                                                                                                                           | —                                                                                                                                                    |
| Document AI          | <b>PyMuPDF</b> (text/layout extraction) + <b>LayoutLMv3</b> (table/section structure) + <b>Tesseract 5 / AWS Texttract</b> (OCR fallback for scans)                                                                                                                                                                       | Matches Blueprint; each tool covers a distinct failure mode (native PDF vs. scanned raster)                                                                                                             | —                                                                                                                                                                                                                                         | —                                                                                                                                                    |
| 3D/BIM export        | <b>IfcOpenShell</b> (IFC4) + <b>ezdxf</b> (DXF)                                                                                                                                                                                                                                                                           | Both are the de facto open-source standard, LGPL/MIT respectively                                                                                                                                       | Proprietary Autodesk Forge API — cost and lock-in with no MVP benefit                                                                                                                                                                     | —                                                                                                                                                    |
| Infra                | <b>Docker + Terraform + AWS (ECS Fargate + RDS + ElastiCache + S3) + Cloudflare</b>                                                                                                                                                                                                                                       | Matches Blueprint's cost model and topology; Fargate avoids EC2 fleet management at MVP scale                                                                                                           | Kubernetes — operational overhead unjustified below Growth horizon                                                                                                                                                                        | Sustained >20 worker nodes or need for custom autoscaling policies (KEDA) → migrate compute workers to EKS, <b>keep the stateless API on Fargate</b> |

## Phase 4 — Repository Structure

|                  |                                                                                          |
|------------------|------------------------------------------------------------------------------------------|
| /apps            |                                                                                          |
| /web             | React SPA (map canvas, HUD, forms, 3D viewport)                                          |
| /api-gateway     | Go service: auth, RBAC, project/site CRUD, job dispatch, WebSocket hub                   |
| /report-renderer | Headless Chromium/Puppeteer PDF service (isolated: no DB write access)                   |
| /workers         |                                                                                          |
| /geometry        | Python: validation, reprojection, boolean ops, buffers (GEOS/Shapely/Clipper2)           |
| /geometry-cgal   | C++ (or Python subprocess wrapper): straight skeleton, LIR, medial axis                  |
| /regulation      | Python: AST rule evaluator (Phase 11) – pure function, no I/O side effects               |
| /layout          | Python: structural bay grid, core placement, MILP unit-packing (OR-Tools)                |
| /parking         | Python: stall packing, ramp geometry, Reeds-Shepp swept-path validation                  |
| /massing         | Python/Node: parametric extrusion mesh generation (feeds Three.js layer data)            |
| /financial       | Python: DCF engine (pure functions – GDV/TDC/NPV/IRR/XIRR/RLV)                           |
| /optimization    | Python: NSGA-II (pymoo) scenario search                                                  |
| /document-ai     | Python: OCR/LayoutLM ingestion pipeline for regulation RAG                               |
| /cad-export      | Python: IfcOpenShell + ezdxf export                                                      |
| /packages        |                                                                                          |
| /domain          | Shared TS + Python entity types generated from one schema source (below)                 |
| /schemas         | JSON Schema (GeoJSON contract, rule DSL) + Zod (TS) + Pydantic (Python) validators       |
| /geo             | Shared geometry constants (SRID tables, UTM zone lookup, NBC dimension constants)        |
| /rules           | Versioned municipal rule JSON files (one file per jurisdiction/version – data, not code) |
| /financial       | Shared rate-card constants, cost-index tables                                            |
| /ui              | Tailwind component library, design tokens                                                |
| /database        |                                                                                          |
| /migrations      | Flyway-numbered SQL (V001_..., V002_...)                                                 |
| /seeds           | Golden-dataset fixture loader (Blueprint's 9 benchmark plots)                            |
| /functions       | PL/pgSQL helper functions (e.g., tenant RLS helper, digest computation)                  |
| /infrastructure  |                                                                                          |
| /docker          | Per-service Dockerfiles + docker-compose.dev.yml                                         |
| /terraform       | AWS: VPC, ECS, RDS, ElastiCache, S3, IAM, Cloudflare DNS                                 |
| /tests           |                                                                                          |
| /unit            | Per-package unit tests (Vitest for TS, PyTest for Python)                                |
| /integration     | Testcontainers-backed Postgres/Redis integration tests                                   |
| /property        | Hypothesis-based geometric fuzzing (Phase 28)                                            |
| /golden          | 9 canonical benchmark plots – regression gate for every geometry/regulation change       |
| /e2e             | Playwright browser flows                                                                 |

**Per-directory rules:** - `/workers/*` **never** import from `/apps/api-gateway` — communication is queue messages + shared `/packages` types only. Prevents the “distributed monolith” failure mode where workers accidentally couple to API internals. - `/packages/rules` holds **data**, never executable logic — this is what makes the regulation engine auditable (a rule change is a JSON diff, not a code deploy). - `/database/migrations` is the only place schema changes are made — no ORM auto-migrate in any service. - `/apps/report-renderer` must not hold direct DB credentials; it receives a pre-signed, already-queried JSON payload from the API gateway, keeping the highest-risk headless-browser process out of the data tier’s trust boundary.

## Phase 5 — Domain Model (delta from Blueprint)

The Blueprint’s entity list (Organization → User → Project → Site → Parcel → Boundary → Road/RoadSegment → Constraint(Tree/Easement/Reservation/Utility) → Regulation/RegulationVersion/Rule/RuleResult → BuildableEnvelope → DevelopmentScenario → Building/Floor/Unit/Core/Stair/Lift/Corridor → Parking → StructuralGrid → Massing → FinancialModel/CostItem/RevenueItem/CashFlow → OptimizationRun/GeneratedOption → Report → AuditEvent) is [DOCUMENTED] with field-level detail already given and is adopted as-is. Three additions close gaps:

- **RuleResult** needs an explicit `evaluation_trace` JSONB column recording which AST nodes fired, in order — required for the “auditable legal citation for every calculation” requirement (Blueprint Deliverable 5/8) and for the RAG-vs-deterministic separation to be provable after the fact, not just asserted.
- **OptimizationRun** needs `population_size`, `generations`, `seed` (integer) columns — NSGA-II is only reproducible if the RNG seed is persisted; the Blueprint’s versioning section requires exact reproducibility and this is the one place a source of nondeterminism can hide.
- **GeneratedOption** needs a `dominance_rank` and `crowding_distance` column (native NSGA-II outputs) so the frontend can render the Pareto frontier without recomputing it.

Soft-delete ( `is_deleted` ) and optimistic-lock ( `version` integer, incremented on every write) are applied to every mutable aggregate root ( `Project` , `Site` , `DevelopmentScenario` ) per the Blueprint’s own schema; PostGIS validity `CHECK` constraints are applied to every geometry column exactly as shown in Blueprint Deliverable 10.

## Phase 6 — PostGIS Engineering

**Rule set (applies to every spatial table):**

1. **Store both representations.** `geom_wgs84` GEOMETRY(..., 4326) for browser round-trip (zero-cost GeoJSON export) and `geom_projected` GEOMETRY(...) in the local UTM zone for every calculation. This is [DOCUMENTED] in Blueprint Deliverable 10/\$5 and is correct — do not calculate area, distance, buffers, or setbacks on the 4326 column; degrees are not a metric unit and Web Mercator (3857) introduces >10% area distortion outside the equator.
2. **UTM zone selection:** derive at site-creation time from the boundary centroid: `zone = floor((lon + 180) / 6) + 1`, hemisphere from latitude sign → EPSG code 326XX (N) or 327XX (S). Store as `srid_projected` INT on `sites` (already in Blueprint schema). Never assume a fixed zone (e.g. 32643) across jurisdictions — the Blueprint’s worked example uses Mumbai/UTM 43N but the platform must generalize.
3. **ST\_Area / ST\_Distance validity:** valid **only** on `geom_projected` for statutory/financial use. `ST_Area(geog, true)` (geography type, spherical) is acceptable for cross-checks (Blueprint’s own regression test does this — planar vs. spherical area within 0.05% tolerance) but must never be the number that feeds an FSI or GDV calculation, since municipal statute defines area in planar/surveyed terms.
4. **Frontend-side calculations are unsafe** for anything statutory: Turf.js area/length on raw WGS84 GeoJSON can be off by several percent and must be labeled `PREVIEW_ONLY` in the UI; every number displayed as “official” must come from a server round-trip.
5. **Generated columns** ( `gross_area_sqm` NUMERIC GENERATED ALWAYS AS ( `ST_Area(boundary_projected)` ) STORED ) per Blueprint — adopt as-is; this eliminates an entire class of stale-cache bug and is the single best schema decision in the source material.
6. **Indexing:** GIST on every geometry column used in a spatial predicate ( `&&` , `ST_Intersects` , `ST_Contains` ); adopt Blueprint’s index list as-is. Add a partial index pattern ( `WHERE NOT is_deleted` ) on every GIST index to keep index size proportional to live data.
7. **Triggers:** `updated_at` auto-touch trigger on every mutable table (standard); a **validity-repair trigger** is explicitly rejected — auto-repairing invalid geometry via `ST_MakeValid` inside a trigger silently changes legally significant boundaries. Validity repair is a worker-initiated, logged, user-visible

operation only (ties to Risk RSK-GEO-01).

8. **Materialized views:** one per jurisdiction for “current effective rule set” ( `mv_active_rules` ), refreshed on rule publish (not on a timer) — regulation changes are rare and event-driven, so timer-based refresh would only introduce staleness risk for no benefit.

## Phase 7 — GeoJSON Contract

Adopt the Blueprint’s canonical `PlatformCanonicalSpatialPayload` JSON Schema (Deliverable 6/§6) verbatim — it already correctly specifies `type`, `metadata{projectId, siteId, sridProjected, timestampUtc, version}`, and per-feature `id/geometry/properties{entityType, source, confidenceScore, statutoryCitation, attributes}`. Two additions:

- Add `regulation_version_id` to feature `properties` for any feature derived from a rule evaluation (buildable envelope, setback line) — without this, a re-rendered historical scenario cannot prove which regulation snapshot produced it.
- Generate the TypeScript (Zod) and Python (Pydantic) validators from **one** JSON Schema source file in `/packages/schemas`, via `json-schema-to-zod` and `datamodel-code-generator` respectively, in a pre-commit/CI step — never hand-maintain three parallel type definitions, which is a guaranteed drift source.

## Phase 8 — Computational Geometry Engine

The Blueprint’s 32–38-row operation matrix (area, buffer, union/intersection/difference, straight skeleton, medial axis, LIR, convex partition, adjacency, etc.) is `[DOCUMENTED]` with correct mathematical definitions and `[EXTERNALLY VERIFIED]` complexity classes and library ownership in this review — GEOS/Shapely for boolean ops and buffers, Clipper2 for high-volume float-precision clipping, CGAL for exact straight-skeleton/LIR/medial-axis, PostGIS as the persistence-layer mirror of GEOS. **No corrections needed to the matrix itself.** What it does not specify is the **service contract**:

### Geometry Worker service contract

|                                                |                                                                                                    |
|------------------------------------------------|----------------------------------------------------------------------------------------------------|
| <code>POST /internal/geometry/validate</code>  | <code>{ geometry } → { valid, reason?, repaired_geometry? }</code>                                 |
| <code>POST /internal/geometry/reproject</code> | <code>{ geometry, from_srid, to_srid } → { geometry }</code>                                       |
| <code>POST /internal/geometry/setback</code>   | <code>{ envelope, edges: [{edge_id, setback_m}] } → { envelope, split_events: [...] }</code>       |
| <code>POST /internal/geometry/buffer</code>    | <code>{ geometry, distance_m, mode: "positive" "negative" } → { geometry, collapsed: bool }</code> |
| <code>POST /internal/geometry/boolean</code>   | <code>{ a, b, op: "union" "intersection" "difference" } → { geometry, is_multi: bool }</code>      |
| <code>POST /internal/geometry/lir</code>       | <code>{ envelope } → { rectangle, width, height, angle }</code>                                    |
| <code>POST /internal/geometry/fragment</code>  | <code>{ geometry, min_area_sqm } → { fragments: [...] }</code>                                     |

Every response carries `engine_version` and `computation_id` (UUID) — these are what get written into `calculation_audit_logs.inputs_digest / outputs_payload` (Phase 26).

**Numeric precision policy** `[PROPOSED]` : snap all incoming vertex coordinates to a 1 mm grid ( `ST_SnapToGrid(geom, 0.001)` ) immediately after reprojection, before any boolean operation — this is the standard mitigation for the “numerical cracking” edge case the Blueprint itself flags in rows 11–13 of the matrix (shared-edge float mismatches producing sliver polygons). Every boolean/buffer/skeleton result is re-validated with `ST_IsValid` before being returned; an invalid result is a **hard worker failure**, surfaced to the user, never silently auto-repaired mid-pipeline (see Risk RSK-GEO-01 mitigation).

**Test strategy per operation (applies uniformly):** (1) unit test against a hand-computed value on a simple shape (rectangle/triangle), (2) property test via Hypothesis generating randomized simple/concave/self-intersecting polygons and asserting invariants (area never increases under difference, area is monotonically non-increasing under negative buffer, etc. — matches Blueprint §30), (3) golden-dataset regression against the 9 benchmark plots (Blueprint §31) on every PR that touches `/workers/geometry*`.

## Phase 9 — Geometry Library Decision (verified)

| Operation class                                                                            | Recommended engine                                                            | Verification note                                                                                                                                                                                                                                                                |
|--------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Polygon boolean (union/intersection/difference)                                            | <b>GEOS</b> (via Shapely) at application layer, <b>PostGIS</b> at query layer | Both wrap the same GEOS C library — no behavioral divergence risk between app and DB layers <code>[EXTERNALLY VERIFIED]</code>                                                                                                                                                   |
| High-volume clipping (many features, tight loop)                                           | <b>Clipper2</b>                                                               | Integer-coordinate internal representation gives faster throughput than GEOS’s double-precision robust predicates for bulk operations (e.g. clipping hundreds of parking stalls against an envelope) <code>[EXTERNALLY VERIFIED]</code>                                          |
| Straight skeleton                                                                          | <b>CGAL</b>                                                                   | GEOS/Shapely have no straight-skeleton implementation as of the versions specified; CGAL’s <code>create_interior_straight_skeleton_2</code> with the exact-predicates-inexact-constructions kernel is the correct, Blueprint-specified choice <code>[EXTERNALLY VERIFIED]</code> |
| Medial axis / LIR / convex partition                                                       | <b>CGAL</b>                                                                   | Same reasoning — these are CGAL-exclusive among the evaluated libraries                                                                                                                                                                                                          |
| Browser preview (non-authoritative)                                                        | <b>Turf.js</b>                                                                | Correct as a <i>preview-only</i> layer; must never be the source of a number shown as final                                                                                                                                                                                      |
| Persistence-layer spatial query ( <code>&amp;&amp;</code> , containment, nearest-neighbor) | <b>PostGIS</b>                                                                | GIST-indexed predicates are an order of magnitude faster than pulling geometry into the app layer for filtering                                                                                                                                                                  |

The Blueprint’s own recommendations here are correct; this table exists to make the decision explicit and testable rather than implicit in worker code.

## Phase 10 — Constraint Engine

Adopt the Blueprint's `site_constraints` schema (category enum, severity enum, `buffer_distance_m` , dual WGS84/projected geometry) as-is. Formalize the severity taxonomy into explicit evaluation behavior, since the Blueprint names the enum but not what each value *does* to the pipeline:

| Severity                   | Pipeline behavior                                                                                                                                                                                          |
|----------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| HARD_ABSOLUTE_EXCLUSION    | Subtracted from the buildable envelope unconditionally (e.g. HT line corridor). Blocks feasibility if it eliminates the envelope entirely.                                                                 |
| CONDITIONAL_NOC_REQUIRED   | Subtracted <b>unless</b> a linked <code>noc_status = APPROVED</code> record exists; otherwise treated as hard exclusion with a <code>warnings[]</code> entry in the API response so the analyst knows why. |
| HEIGHT_RESTRICTION_SURFACE | Does not reduce footprint; instead clips <code>max_height_m</code> on the <code>BuildableEnvelope</code> (e.g. airport funnel surface, HT clearance-by-height).                                            |
| SETBACK_OFFSET_MODIFIER    | Feeds into the straight-skeleton setback stage as an additive/multiplicative modifier on the statutory setback for the affected edge, rather than being an independent subtraction.                        |

`effective_from` / `effective_to` are added to `site_constraints` [PROPOSED] so conditional constraints (e.g. a temporary NOC, a road-widening notification with a sunset date) can expire without a manual delete, keeping the audit trail intact.

## Phase 11 — Regulation Engine: Rule DSL and Deterministic Evaluator

The Blueprint fully specifies the **rule JSON schema** (jurisdiction/authority/regulationCode/version/roadWidthRules[]/peripheralSetbackFormula/parkingNorms) and gives a correct, usable Mumbai DCPR 2034 example — this is adopted verbatim as the data format. **What neither Blueprint document specifies is the evaluator itself** — the thing that turns that JSON into a `RuleResult` . This is designed here.

### Evaluator architecture



**Why an AST and not “just evaluate the JSON with if/else in application code”:** road-width band lookups, premium/TDR eligibility, and peripheral setback formulas ( `MAX(3.0, heightM/5.0)` ) compose differently per jurisdiction — Bengaluru’s setback is 8% of plot depth , Mumbai’s is a flat table lookup, Hyderabad’s is `height/3` with a cap. Hard-coding this as imperative per-city code means every new jurisdiction is a code deploy with regression risk to existing cities. An AST compiled from a declarative schema means a new jurisdiction is a **data file**, reviewed and published by a municipal-architect user through the Human Verification Gate, with zero application-code change — this is the load-bearing design decision the “declarative, version-controlled” claim in the Blueprint depends on, and it does not work without an evaluator built this way.

**AST node types (minimum viable set for the jurisdictions in the Blueprint’s matrix):** - `BandLookup(input_field, bands[{min, max, output}])` — road-width/FSI banding (all 7 states use this shape). - `Formula(expr_string, variables)` — safely-evaluated arithmetic ( `height/3, 8%*plot_depth` ) via a restricted expression parser ( `asteval` or a hand-rolled recursive-descent parser over `+ - * / MAX MIN` only) — **never** `eval()` on user- or document-derived strings; this is a direct RSK-REG-02 mitigation, since an unsandboxed evaluator is also an injection surface if a rule is ever authored from OCR’d text. - `Conditional(predicate, then, else)` — e.g. high-rise vs. low-rise branch above/below the 15 m NBC threshold. - `Citation(document, clause, gazette_ref)` — attached to every leaf node so `evaluation_trace` can produce the “Source: Maharashtra UD CPR 2020, Chapter 6, Table 6-G, Clause 6.3(b)” string the Blueprint’s worked example requires.

**Determinism guarantee:** the evaluator is a pure function — same ( `rule_version_id, inputs` ) always produces the same `RuleResult` , which is exactly the reproducibility property the versioning system (Phase 26) depends on. No network call, no clock read, no random number inside the evaluator.

**No LLM-generated number reaches this evaluator as an input or is treated as its output** — the AI Copilot (Phase 22) calls this evaluator as a tool and only narrates its output; it never substitutes for it. This closes the Blueprint’s own stated risk (RSK-REG-02) at the architecture level rather than by policy alone.

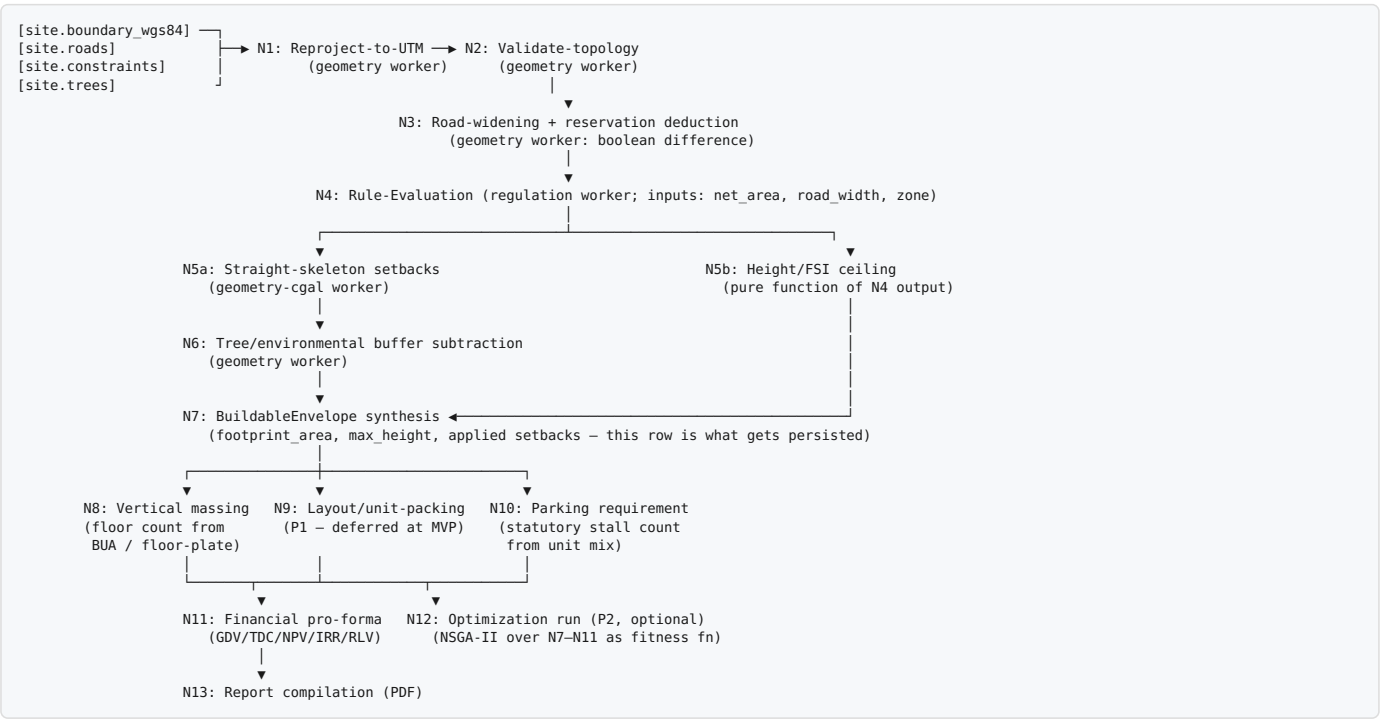
## Phase 12 — Regulation RAG (ingestion pipeline)

Adopt the Blueprint’s pipeline as the correct shape: PDF → OCR (Tesseract/Texttract) → Layout detection (LayoutLMv3 + Table-Transformer) → Section/clause extraction → Metadata tagging (jurisdiction, authority, document, chapter, clause, gazette date) → Embedding (text-embedding-3-large) → Qdrant (dense vector search) + Neo4j (clause dependency graph: `DEPENDS_ON` / `EXEMPTS` / `SUPERSEDES`) → Confidence scoring → Human Verification Gate → Published Rule JSON (feeds Phase 11) .

**Hard boundary [DOCUMENTED, reinforced]** : the RAG system’s output is *never* written directly into `/packages/rules/*.json` . It produces a `PENDING_REVIEW` row that a licensed architect/planner must approve; approval is what triggers the Rule Compiler to pick it up. A RAG **answer** (natural-language, cited, for the copilot) and a **published rule** (feeds the deterministic evaluator) are two different tables ( `rag_answers` vs. `rule_definitions` ) with no code path that promotes one into the other automatically. Confidence threshold  $\sigma < 0.95$  routes to mandatory human review per the Blueprint — adopted as-is; below  $\sigma < 0.70$  the extraction is discarded rather than queued, to keep the review queue signal-to-noise workable.

## Phase 13 — Feasibility Engine as an Executable DAG

The Blueprint describes the feasibility pipeline as an ordered sequence of formulas (Deliverable 13/§10). Here it is formalized as an actual DAG the orchestrator executes, since “ordered sequence” alone does not specify what happens on partial failure, what gets cached, or what triggers recomputation.



**Orchestrator contract:** each node is a queued job with (node\_id, scenario\_id, input\_hash) as its idempotency key; re-running the same inputs returns the cached RuleResult / BuildableEnvelope row rather than recomputing (this is what makes “every calculation reproducible” operationally true, not just a design intent). A failure at any node marks that node and all downstream nodes **BLOCKED** in the job-status table and surfaces the specific failing node to the UI — partial results (e.g., a valid BuildableEnvelope with parking not yet computed) are shown rather than the whole run failing opaquely. N1-N7 run on the **interactive/high-priority queue** (target P95 < 2 s, matching Blueprint’s own auto-save SLA); N8-N13 run on the **batch queue**.

## Phase 14 — Generative Layout Engine & Unit Packing

Adopt the Blueprint’s six-stage pipeline (footprint ingestion → structural bay grid → core placement → corridor routing → unit demising → room decomposition) and its NBC 2016 dimensional constants (Deliverable 6/§12) verbatim — they are correctly sourced and match the National Building Code.

**Algorithm choice per subproblem** [PROPOSED], validated against the Blueprint’s own comparison table:

| Subproblem                                                                 | Technique                                                                                              | Why this and not the alternatives                                                                                                                                                                                                                                                                 |
|----------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Structural bay grid                                                        | Deterministic heuristic (regular grid snapped to envelope longest edge, span 6.0–8.4 m)                | No competing objectives — a regular grid is provably near-optimal for span efficiency; MILP would be over-engineering for a single-objective placement                                                                                                                                            |
| Core placement (stairs/lifts/shafts)                                       | Constraint-based placement near centroid, minimizing max egress distance                               | A small, discrete search space (few candidate positions) — MILP is unnecessary; simulated annealing is unnecessary; direct centroid-distance minimization with fire-code constraints as hard filters is sufficient and fast                                                                       |
| Corridor routing                                                           | Shortest-path spine connecting core to perimeter, dead-end capped at 6.0 m                             | Graph shortest-path (Dijkstra on a visibility graph derived from the envelope) — deterministic and auditable                                                                                                                                                                                      |
| Unit demising (perimeter → discrete apartment volumes matching target mix) | <b>MILP</b> (OR-Tools CBC/SCIP)                                                                        | This is genuinely combinatorial — partitioning a perimeter into N discrete unit types under an area-ratio target and adjacency constraints (wet-stack alignment) is the textbook MILP use case the Blueprint correctly identifies; heuristics here would leave value (saleable area) on the table |
| Internal room decomposition                                                | Constraint satisfaction (rule-based placement respecting NBC minimums + daylight-frontage requirement) | A small per-unit search space with hard dimensional constraints — CSP solvers (e.g. python-constraint or a bespoke backtracking solver) are appropriate; full MILP per-room is unnecessary compute for the value gained                                                                           |

**MILP formulation sketch for unit demising:** decision variables  $x_{i,t} \in \{0,1\}$  (unit slot  $i$  assigned type  $t$ ), objective  $\text{maximize } \sum \text{saleable\_area}(t) \cdot x_{i,t}$ , constraints:  $\sum_t x_{i,t} = 1 \ \forall i$  (each slot exactly one type),  $\sum_i x_{i,t} / N \approx \text{target\_mix}[t] \pm \text{tolerance}$  (mix ratio), wet-stack alignment as a same-column constraint between vertically adjacent floors’ slot assignments, minimum frontage per unit type from NBC table. This is directly implementable with `ortools.linear_solver`.

**Output is vector, not raster** — every generated unit/room is a Unit/geometry row with `boundary_projected`, never a bitmap, matching the Blueprint’s explicit “not raster images” requirement, and enabling downstream IFC/DXF export.

## Phase 15 — Parking / Mobility Engine

Adopt the Blueprint’s stall dimensions (2.5×5.0 m standard, 3.6×5.0 m ADA), aisle widths by angle (90°/60°/45°), ramp gradients (1:8 straight, 1:10 curved, 1:12 pedestrian), and the kinematic bicycle model for swept-path validation (Deliverable 13, §13) — all correct and directly implementable.

**Layout algorithm:** bin-packing of the stall grid against the available podium/basement footprint (greedy row-packing along the long axis, then aisle insertion at the computed module width), followed by a **Reeds-Shepp swept-path validation pass** that checks every turning maneuver (entry, internal turns, ramp transitions) against the packed geometry — a candidate layout that fails swept-path validation at any turn has that row/bay removed and is re-packed, iterating until a feasible layout is found or the footprint is exhausted. This is a constructive-then-verify approach rather than a joint optimization, which is appropriate because parking stall count is a statutory *minimum* to satisfy, not a value to maximize — once the minimum is met and swept-path clears, further optimization has low marginal value versus a MILP formulation’s added complexity.

Fire-tender access (Blueprint’s 6.0 m driveway, 9.0 m turning radius, 45.0 m max dead-end) is validated by the same swept-path check using the larger fire-tender wheelbase (4.5–6.0 m vs. 2.8 m passenger car) as a separate, higher-priority pass — a layout that satisfies passenger-car swept paths but fails fire-tender swept paths is rejected outright, never offered as a “reduced fire access” option.

---

## Phase 16 — Structural Grid (conceptual only)

Adopt the Blueprint’s explicit framing: this module produces **advisory** column-grid and shear-wall-zone suggestions (spans 6.0–8.4 m parking-optimized / 3.0–4.2 m residential sub-grid), aligned to the layout engine’s unit demising output, and is labeled in the UI and in every export as “Conceptual — not licensed structural design.” It must never be the sole basis for a structural drawing; the platform’s export (IFC/DXF) carries this disclaimer in document metadata, and no API response from this module is described as “structural design” anywhere in the frontend copy. This distinction is a genuine liability boundary and is treated as a hard product requirement, not a nice-to-have caveat.

---

## Phase 17 — 3D Massing Engine

Adopt the Blueprint’s Three.js-custom-MapLibre-layer implementation (Deliverable 7) essentially verbatim — the `CustomLayerInterface` code shown correctly shares a single WebGL context between the 2D map and 3D massing (avoiding `CONTEXT_LOST_WEBGL`), correctly disposes geometry/material on `onRemove`, and is current against the Three.js r160+ API. Additions:

- **Scene graph:** one `THREE.Group` per `Building`, one child mesh per `Floor` (extruded from that floor’s footprint polygon via `ExtrudeGeometry`), instanced geometry for repeated elements (parking stall markers, balcony modules) via `THREE.InstancedMesh` to keep draw calls bounded as unit count grows.
- **LOD:** at zoom levels below building-scale, render a single extruded envelope mesh (the `BuildableEnvelope`, not per-floor); floor-level detail loads only above a zoom threshold — this keeps regional/multi-parcel views performant without needing Cesium.
- **Coordinate conversion:** massing meshes are built in the site’s local UTM metric frame, then transformed into Mapbox/MapLibre’s Mercator unit space inside the custom layer’s `render()` call using the projection matrix supplied by the map — never bake Mercator coordinates into the geometry buffers themselves, since that breaks re-use of the same mesh data for IFC/DXF export (which needs metric coordinates).
- **GPU performance guardrail:** cap total triangle count per scene (e.g. 500k) with an automatic floor-detail LOD downgrade above that threshold, to keep the browser stable on mid-range hardware during multi-building master-plan views.

---

## Phase 18 — Financial Feasibility Engine

Adopt the Blueprint’s full formula set verbatim — GDV, TDC (with its `C_land / C_statutory / C_hard / C_soft / C_finance / C_contingency` decomposition), NPV, IRR, XIRR, RLV — these are standard, correctly stated institutional real-estate underwriting formulas (Deliverable 8/§17), independently checked against standard DCF definitions. **[EXTERNALLY VERIFIED]**.

**Implementation requirement not stated in the Blueprint:** every formula is implemented as a **pure function** (`compute_gdv(inputs) -> GDV`, no hidden state, no DB read inside the function) in `/workers/financial`, unit-tested against the Blueprint’s own worked example (Deliverable 20 — the Andheri East parcel produces  $GDV \approx ₹48.06$  Cr,  $TDC \approx ₹39.19$  Cr,  $IRR \approx 24.85\%$ ) as a golden regression test with **zero tolerance** (financial formulas must match audited spreadsheet output exactly, per the Blueprint’s own testing-strategy requirement). IRR/XIRR are solved via Newton-Raphson with a documented fallback to bisection on non-convergence (IRR has no closed form and can have zero or multiple real roots for irregular cash-flow sign patterns — the fallback and its failure mode must be surfaced to the user as “IRR undefined for this cash-flow pattern,” never silently returned as `0` or `null`).

---

## Phase 19 — Scenario Engine

A `DevelopmentScenario` is forked from a baseline (or another scenario) by copying its input parameters (unit mix, FSI strategy, structural strategy) into a new row with a new `id` and `parent_scenario_id` **[PROPOSED addition]** — the DAG (Phase 13) then re-executes from whichever node the changed input first affects (e.g. changing `target_unit_mix` only re-triggers N9 onward, not N1–N8, since geometry and statutory envelope are unaffected). This selective re-execution is what the Blueprint’s “immutable calculation runs” requirement demands in practice — a full-DAG re-run on every scenario tweak would be both slow and would falsely invalidate unrelated cached results. Every scenario run is immutable once computed (no in-place update to a `BuildableEnvelope` or `FinancialModel` row — a new version row is written, matching Phase 26’s versioning model), so two scenarios can always be compared side-by-side by reading two distinct row sets.

---

## Phase 20 — Multi-Objective Optimization Engine (NSGA-II)

Adopt the Blueprint’s objective set (maximize saleable area, revenue, profit, daylight; minimize cost, structural complexity, circulation, regulatory risk — trimmed to the 6 objectives named in the formal problem statement of the second Blueprint, Deliverable/§18) and its constraint set (statutory FSI ceiling, envelope containment, parking minimums, fire-tender clearance) — correct as the problem statement.

**Chromosome encoding** **[PROPOSED – not specified by either Blueprint]** :

```

gene[0]      : footprint_offset_x, footprint_offset_y  (continuous, bounded by envelope)
gene[1]      : tower_rotation_theta                  (continuous, 0–180°)
gene[2]      : podium_height_floors                   (integer)
gene[3]      : tower_height_floors                    (integer, bounded by FSI/height cap)
gene[4]      : structural_bay_spacing                 (continuous, 6.0–8.4 m)
gene[5..N]   : unit_mix_ratios                       (continuous, simplex-constrained, sum=1)

```

Real-valued genes use simulated binary crossover (SBX) and polynomial mutation ( `pymoo` defaults); integer genes use the library’s integer-variable handling rather than rounding continuous genes post-hoc (rounding after crossover is a common source of constraint violation in naive GA implementations). **Hard-constraint violations are handled via constraint domination** (a feasible solution always dominates an infeasible one, per Deb’s original NSGA-II constrained-domination rule), not via a penalty term added to the fitness — penalty terms require hand-tuned weights that silently bias the Pareto frontier; constraint domination does not.

**Reproducibility:** `population_size`, `n_generations`, and an explicit integer `seed` are required run parameters (Phase 5 schema addition) and are persisted on `OptimizationRun`; **stopping criterion** is `n_generations` fixed at run start (not a convergence heuristic) so that “re-run this exact scenario” is well-defined and produces bit-identical output given the same seed and library version — `engine_version` is recorded for exactly this reason (a `pymoo` version bump can change RNG draw order even with the same seed, so historical runs record which version produced them).

## Phase 21 — AI Development Copilot

Adopt the Blueprint’s architecture verbatim: USER → LLM (intent parsing) → typed tool call → deterministic engine → JSON result → LLM (explanation synthesis, no number modification) . This is the correct pattern and is treated as non-negotiable — the LLM never computes a setback, an FSI figure, or a financial number itself; it only selects and narrates tool output.

**Tool schemas (typed, matching the Blueprint’s named tools):**

```

calculate_feasibility(site_id, scenario_id) -> FeasibilitySummary
evaluate_rule(site_id, jurisdiction_code) -> RuleResult           # calls Phase 11 evaluator
calculate_area(geometry) -> { area_sqm, method: "planar"|"geodesic" }
generate_layout(scenario_id, unit_mix) -> JobHandle             # queues N9
calculate_parking(scenario_id) -> ParkingSummary
run_financial_model(scenario_id, assumptions) -> FinancialSummary # calls Phase 18 engine
compare_scenarios(scenario_ids: [uuid]) -> ComparisonTable
generate_report(scenario_id, format) -> JobHandle                # queues N13

```

Each tool call is logged with the same `calculation_id / engine_version` pattern as Phase 26, so an AI-initiated calculation is exactly as auditable as a UI-initiated one — there is no separate, less-audited code path for AI-triggered work. The LLM’s system prompt explicitly forbids stating any numeric planning value that did not come from a tool result in the current turn, and the explanation-synthesis step is instructed to quote the tool’s `citation` field rather than paraphrase a statutory clause from its own training knowledge.

## Phase 22 — API Specification (OpenAPI 3.1, key contracts)

Adopt and complete the Blueprint’s endpoint table (Deliverable 11/22). Two representative full contracts (the rest follow the same shape — request schema in `/packages/schemas`, 202 Accepted + `jobId` for anything that queues a worker, 200 / 201 for synchronous CRUD):

```

POST /api/v1/projects
  auth: Organization Member (Bearer, short-lived JWT)
  idempotency: Idempotency-Key header required
  body: { title, description?, landTenure }
  201 -> { projectId, status: "DRAFT" }
  400 -> validation error (Zod/Pydantic schema violation)
  409 -> Idempotency-Key reused with different body

POST /api/v1/feasibility/runs
  auth: Project Editor
  idempotency: yes (runId supplied by client, UUID)
  body: { siteId, scenarioId }
  202 -> { jobId, status: "QUEUED", dagNodes: ["N1","N2",...] }
  404 -> site or scenario not found / not in caller's org (RLS-enforced, same response either way – no tenant-existence leakage)
  429 -> per-org concurrent-run limit exceeded

GET /api/v1/jobs/:jobId
  auth: Job Owner
  200 -> { jobId, status: "QUEUED"|"RUNNING"|"BLOCKED"|"COMPLETED"|"FAILED",
    completedNodes: [...], failedNode?: { nodeId, reason }, progressPct }

WS /api/v1/realtime/connect
  auth: verified session (upgrade from authenticated HTTPS session, not a bearer token in the URL)
  server -> client events: job.progress, job.completed, scenario.updated, presence.cursor

```

All mutating endpoints require `Idempotency-Key` ; all list/get endpoints are cacheable via `TanStack Query` . Rate limiting is enforced at the Kong/Envoy ingress layer (Blueprint Tier 2), not in application code, so the limit is uniform across every service behind it. Every `4xx / 5xx` body follows one error envelope shape ( { status: "ERROR", code, message, details } ), matching the Blueprint’s own example in §38) so the frontend has one error-handling code path.

## Phase 23 — Worker Architecture

| Worker pool                            | CPU/RAM (baseline)                        | Runtime                                         | Queue priority                                                     | Timeout                                          | Idempotency key                   |
|----------------------------------------|-------------------------------------------|-------------------------------------------------|--------------------------------------------------------------------|--------------------------------------------------|-----------------------------------|
| Geometry (GEOS/Shapely/Clipper2)       | 1 vCPU / 1 GB                             | Python 3.12                                     | Interactive (high)                                                 | 10 s                                             | (node_id, input_hash)             |
| Geometry-CGAL (straight skeleton, LIR) | 2 vCPU / 2 GB                             | C++ (subprocess) or gRPC service                | Interactive (high)                                                 | 15 s                                             | (node_id, input_hash)             |
| Regulation (AST evaluator)             | 0.5 vCPU / 512 MB                         | Python 3.12                                     | Interactive (high)                                                 | 2 s (pure function — should never approach this) | (rule_version_id, input_hash)     |
| Layout/MILP                            | 2 vCPU / 4 GB                             | Python 3.12 + OR-Tools                          | Batch                                                              | 120 s                                            | (scenario_id, params_hash)        |
| Parking                                | 1 vCPU / 1 GB                             | Python 3.12                                     | Batch                                                              | 60 s                                             | (scenario_id, params_hash)        |
| Financial (DCF)                        | 0.5 vCPU / 512 MB                         | Python 3.12                                     | Interactive (high — cheap, users expect instant pro-forma updates) | 3 s                                              | (scenario_id, assumptions_hash)   |
| Optimization (NSGA-II)                 | 4 vCPU / 4 GB                             | Python 3.12 + pymoo                             | Batch (long)                                                       | 1800 s                                           | (scenario_id, seed, params_hash)  |
| Document-AI (OCR/LayoutLM)             | 2 vCPU / 4 GB (GPU optional for LayoutLM) | Python 3.12                                     | Batch                                                              | 300 s                                            | (document_id, page_hash)          |
| Report/Export (Chromium PDF, IFC/DXF)  | 1 vCPU / 2 GB                             | Node (Puppeteer) / Python (IfcOpenShell, ezdxf) | Batch                                                              | 60 s                                             | (scenario_id, report_config_hash) |

All queues are Celery-on-Redis at MVP, partitioned by priority (interactive vs. batch) exactly as the Blueprint’s Tier 4 describes, with a **retry policy** of 3 attempts with exponential backoff for transient failures (DB connection blips) and **zero retries** for deterministic-failure classes (invalid geometry, constraint violation) — retrying a deterministic failure just wastes compute and delays the user-visible error. Every worker publishes a heartbeat/progress percentage to Postgres, which the API gateway’s WebSocket hub relays to the client (Blueprint’s “Progress Notification Stream”).

## Phase 24 — Frontend Architecture

Adopt the Blueprint’s stack and pattern (React 18/19 + TypeScript, TanStack Query v5 for server state, Zustand for ephemeral UI state, MapLibre + Mapbox Draw plugin compatibility, `useRef` -isolated map canvas to prevent React reconciliation from destroying the WebGL context, debounced auto-save via `AbortController`) — all [DOCUMENTED] and correct as written.

**Pages** (per Blueprint’s list): Dashboard, Projects, Project Workspace, Map, Site Analysis, Constraints, Regulations, Feasibility, Layouts, Floor Plans, 3D Massing, Parking, Financial, Scenarios, Optimization, Reports, Settings.

**State architecture:** - **Server state (TanStack Query):** projects, sites, scenarios, rule results, financial models, job status — anything the API owns. Query keys are hierarchical ( `['project', id]` , `['project', id, 'scenarios']` ) so a scenario mutation can precisely invalidate only its own subtree. - **Client UI state (Zustand):** active draw tool, selected vertex IDs, modal visibility, sidebar tab, camera position (mirrored to URL, see below). - **URL state:** `projectId` , `scenarioId` , `map lat/lng/zoom` — enables shareable deep links (Blueprint’s own example: `/projects/:id?scenario=B&lat=19.115&lng=72.850&zoom=16` ). - **Undo/redo:** implemented as a command stack over geometry-edit operations only (not global app state) — each command is `{ do(), undo() }` operating on the Zustand draw-state slice; server persistence happens on debounce independent of the undo stack, so undo/redo is instantaneous and local while auto-save remains eventually-consistent with the server. - **Offline queue:** mutations attempted while offline are appended to an IndexedDB-backed queue (not the Blueprint’s original localStorage) and flushed in order on reconnect; each queued mutation carries the same `Idempotency-Key` it would have used online, so a flush that partially succeeded before a second disconnect never double-applies. - **WebSocket updates:** job-progress and scenario-updated events invalidate the relevant TanStack Query keys rather than pushing raw data into the cache directly — this keeps TanStack Query as the single source of truth for what’s rendered and avoids two competing state-update paths.

## Phase 25 — CAD-Grade Map Editor

Adopt the Blueprint’s interaction spec verbatim (Deliverable 24/\$24): vertex/edge/midpoint snapping via a client-side Flatbush R-tree spatial index with a 12-pixel screen-space tolerance, Shift-held orthogonal locking at 0/45/90/135/180°, heads-up numeric length/angle entry, edge-splitting on midpoint click, and real-time self-intersection detection that turns the in-progress edge red and blocks vertex placement. Chamfer/fillet and boolean edit operations (union/difference on drawn shapes) call the same `/internal/geometry/*` worker endpoints as Phase 8, run synchronously on the interactive queue given their sub-2-second SLA requirement, and return the edited geometry for immediate canvas re-render — client-side Turf.js is used only for the *preview* stroke while dragging, with the authoritative shape coming from the server round-trip on mouse-up, consistent with the “frontend calculations are unsafe for statutory numbers” rule from Phase 6.

## Phase 26 — Versioning

Every `BuildableEnvelope` , `RuleResult` , `FinancialModel` , and `OptimizationRun` row carries: `project_version` , `site_version` , `geometry_version` (hash of the input boundary/constraints), `regulation_version_id` , `financial_version` , `engine_version` , `created_at` , and a SHA-256 `inputs_digest` — matching the Blueprint’s `calculation_audit_logs` table design (Deliverable 10) which is adopted as-is. **Immutability is enforced at the database level**, not just by convention: a BEFORE UPDATE trigger on `calculation_audit_logs` (and on `buildable_envelopes` , `financial_models` once they exist as tables) rejects any UPDATE statement outright — the only legal operation is INSERT of a new versioned row [PROPOSED – closes a gap in the Blueprint’s schema, which defines the audit table but not the immutability enforcement mechanism]:

```
CREATE OR REPLACE FUNCTION reject_update() RETURNS trigger AS $$
BEGIN RAISE EXCEPTION 'Immutable table: % rows cannot be updated', TG_TABLE_NAME; END;
$$ LANGUAGE plpgsql;
CREATE TRIGGER no_update_calculation_audit_logs
BEFORE UPDATE ON calculation_audit_logs
FOR EACH ROW EXECUTE FUNCTION reject_update();
```

A previous feasibility result is reproduced by re-invoking the DAG orchestrator with the historical `(geometry_version, regulation_version_id, engine_version)` triple pinned — the orchestrator reads the pinned rule JSON file for that exact version (never “latest”) and the pinned worker container image tag for that `engine_version`, which is why rule files are versioned, immutable files in `/packages/rules` rather than mutable database rows.

## Phase 27 — Testing

Adopt the Blueprint’s multi-tier strategy (Vitest/PyTest unit tests, Hypothesis property-based geometric fuzzing, Testcontainers-backed Postgres/PostGIS integration tests with RLS assertions, Playwright E2E including visual-regression canvas snapshots, financial golden-benchmark tests at zero tolerance against audited spreadsheets) — this is correct and complete as a strategy; adopted verbatim.

**Concrete CI gate structure [PROPOSED]** : 1. Lint + typecheck (fails fast, <1 min) 2. Unit tests, all packages, parallelized 3. Golden-dataset regression (the 9 benchmark plots, Phase 8/§31) — **required, blocking**, run on every PR touching `/workers/geometry*`, `/workers/regulation`, or `/packages/rules` 4. Property-based fuzzing (Hypothesis, geometry invariants) — required on the same trigger paths, 1000+ generated cases per run 5. Integration tests (Testcontainers Postgres, RLS cross-tenant isolation assertions) — required on every PR touching `/database` or `/apps/api-gateway` 6. Financial golden benchmark (zero-variance against the Blueprint’s Andheri East worked example) — required, blocking, on every PR touching `/workers/financial` 7. Playwright E2E (login → geocode → draw boundary → assert debounced auto-save < 1000 ms → assert 3D canvas renders) — required on every merge to `main`, not every PR (cost/time tradeoff) 8. Visual regression (canvas pixel-diff against baseline) — nightly, not per-PR, to avoid flaking on font/anti-aliasing noise

Golden-dataset regression and financial zero-variance are the two gates the Blueprint itself singles out as non-negotiable (RSK-REG-02, RSK-GEO-01), so they are the only two given hard **zero-tolerance, always-blocking** status here — everything else uses normal test-flake tolerance.

## Phase 28 — Security

Adopt the Blueprint’s security architecture in full — it is [EXTERNALLY VERIFIED] correct and specific enough to implement directly: replace `localStorage` JWT with `HttpOnly/SameSite=Strict` cookies + 15-minute access token + rolling refresh rotation; PostgreSQL Row-Level Security keyed on `organization_id` via `current_setting('app.current_org_id')`, with the exact `CREATE POLICY` statement given in Deliverable 28/§28 adopted verbatim; the CSP header exactly as specified (`default-src 'self'; script-src 'self'; worker-src 'self' blob;; child-src blob;; connect-src 'self' https://api.mapbox.com https://*.tiles.mapbox.com wss://*.<domain>; img-src 'self' data: blob: https://api.mapbox.com;` ); TLS 1.3 in transit; AES-256 with tenant-scoped KMS keys at rest.

**Additions closing gaps the Blueprint names but doesn’t fully specify**: - File-upload security (cadastral survey/document ingestion): virus-scan on upload (ClamAV in the upload worker), strict MIME-type allowlist, file-size cap, and uploaded documents are never directly executed/rendered server-side by anything with DB credentials — the document-AI worker pool is the only consumer, and it runs with read-only, time-limited S3 access, not a persistent credential. - API rate limiting is per-organization **and** per-user (not just global), preventing one compromised account inside a large org from exhausting the whole tenant’s quota. - Every worker container runs with a read-only root filesystem and no outbound network access except to Redis/Postgres/S3 endpoints explicitly allow-listed at the security-group level — closes a class of exfiltration risk in the compute tier that handles the most commercially sensitive data (land pricing, pro-formas).

## Phase 29 — Observability

Adopt OpenTelemetry (distributed tracing), Prometheus/Grafana (metrics), and structured audit logging as named in the Blueprint. **Correlation contract [PROPOSED, closes the gap the Blueprint leaves open]**: every request entering the API gateway is assigned a `correlation_id` (UUID) at the ingress; this ID is propagated into every queued job’s message envelope and every worker’s log line, and is written onto the `calculation_audit_logs.triggered_by`-adjacent `correlation_id` column — so a single trace connects the HTTP request, every DAG node’s worker execution, and the resulting audit row. Tracked metrics: API latency (p50/p95/p99) per endpoint, per-node worker execution duration, queue depth per pool, geometry operation failure rate (by operation type — this is the earliest signal of a Risk RSK-GEO-01 regression), rule-evaluation duration, optimization-run wall-clock time, WebGL frontend crash rate (via a client-side error boundary reporting to the same telemetry pipeline), and database query p99 on the GIST-indexed spatial predicates specifically (a regression here usually means an index was dropped or a query stopped using it).

## Phase 30 — Deployment

Adopt the Blueprint’s AWS topology (Cloudflare edge → ALB across 3 AZs → ECS Fargate stateless API fleet → ElastiCache Redis → EC2 compute-optimized worker nodes → RDS Multi-AZ PostgreSQL+PostGIS → S3) as the target for MVP through Growth horizons.

**Environments**: Local (docker-compose, all services + Postgres/Redis containers), Development (shared AWS account, single-AZ, scaled-down instance sizes, auto-torn-down after 14 days of inactivity), Staging (mirrors Production topology at 1/4 scale, used for the golden-dataset and Playwright E2E gates pre-release), Production (Multi-AZ as specified).

**CI/CD**: GitHub Actions (or equivalent) — PR pipeline runs Phase 27’s gates 1–6; merge-to-main additionally runs gate 7 and deploys to Staging automatically; Production deploy is a manual-approval promotion of a Staging-verified image tag (never a fresh build) — this guarantees the exact artifact that passed E2E is what ships, closing a common CI/CD correctness gap.

Database migrations run via Flyway as a separate pipeline step **before** the new application image is deployed, with every migration required to be backward-compatible with the *previous* application version for one release cycle (additive-only: new nullable columns, new tables — no drops/renames without a two-step migrate-then-cleanup pattern), enabling zero-downtime rolling deploys on Fargate.

**Kubernetes is explicitly not introduced** at MVP or Growth horizons per the architecture validation in Phase 2; the stated trigger (>20 sustained worker nodes

or a KEDA-specific autoscaling need) is the only condition under which the compute-worker tier migrates to EKS — the stateless API tier has no stated trigger to leave Fargate at all, since its scaling profile (HTTP concurrency) is exactly what Fargate is designed for.

## Phase 31 — Cost Model

Adopt the Blueprint’s cost table (Deliverable 33) as the reference model — it is internally consistent (line items sum correctly to the stated totals) and its per-tier scaling assumptions (RDS instance class, Fargate task count, worker EC2 fleet size, S3/CDN egress, LLM token spend, Mapbox/MapLibre tile cost, observability tooling cost) are reasonable for the stated project/org volumes: **≈\$470/mo at MVP (<100 projects)**, **≈\$2,240/mo at Growth (1,000 projects/250 orgs)**, **≈\$8,150/mo at Enterprise (10,000 projects/2,500 orgs)**, **≈\$28,400/mo at Institutional scale (100,000 projects)**. The dominant cost driver at scale (optimization/geometry worker compute + satellite tile consumption) matches the architectural reality that NSGA-II and CGAL straight-skeleton runs are the platform’s heaviest CPU consumers — no revision needed to this model beyond noting that adopting MapLibre (Phase 3) instead of paid Mapbox tiles can reduce the “Mapbox/MapLibre Cartography” line item toward its \$0 floor if a self-hosted or OSM-derived basemap is acceptable for the target markets.

## Phase 32 — Risk Register

Adopt the Blueprint’s six named risks (RSK-GEO-01 degenerate polygon ingestion, RSK-REG-02 statutory hallucination, RSK-REG-03 bye-law lag, RSK-WGL-04 WebGL memory leaks, RSK-SEC-05 cross-tenant data leaks, RSK-FIN-06 construction cost inflation) verbatim with their stated severity/likelihood/mitigation — all are well-formed and their mitigations are already reflected in the phases above (RSK-GEO-01 → Phase 8/27, RSK-REG-02 → Phase 11/21, RSK-SEC-05 → Phase 28). Additions covering categories the Blueprint’s prompt scope names but the risk register itself omits:

| Risk ID   | Description                                                                                                                                                                                                             | Severity | Likelihood | Mitigation                                                                                                                                                                                                                                                                                               |
|-----------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| PERF-01   | Long-running geometry op (pathological polygon: thousands of vertices, near-degenerate angles) exceeds worker timeout, blocking the interactive queue                                                                   | High     | Medium     | Hard vertex-count cap on ingestion (simplify via Douglas-Peucker above N vertices before entering the pipeline); per-node timeout (Phase 23 table) with graceful <b>BLOCKED</b> status rather than worker crash; circuit-breaker moving repeatedly-timing-out job types to the batch queue automatically |
| OPT-01    | NSGA-II run fails to converge to a useful Pareto front within the fixed generation budget (poorly conditioned constraint set, e.g. envelope too small for any feasible unit mix)                                        | Medium   | Medium     | Fixed generation count (Phase 20) bounds worst-case runtime regardless of convergence; if the final population contains zero feasible individuals, the API returns an explicit <b>INFEASIBLE</b> result with the binding constraint named, never an empty/silent success                                 |
| WORKER-01 | Duplicate job execution (message redelivery after a worker crash mid-task, or a retry racing a slow-but-succeeding original attempt)                                                                                    | Medium   | Medium     | Every job’s idempotency key (Phase 23 table) makes duplicate execution a no-op on write — the second writer’s <b>INSERT</b> either matches the existing row’s digest (safe no-op) or is rejected by the immutability trigger (Phase 26), never silently overwrites                                       |
| UX-01     | User navigates away or loses connectivity mid-draw, before the debounced auto-save fires, losing unsaved geometry edits                                                                                                 | Medium   | High       | Offline/pending-write queue in IndexedDB (Phase 24) persists across a tab reload; a <b>beforeunload</b> warning fires whenever the pending-write queue is non-empty                                                                                                                                      |
| FIN-02    | IRR/XIRR solver fails to converge (irregular cash-flow sign pattern with zero or multiple real roots)                                                                                                                   | Low      | Low        | Explicit <b>INDETERMINATE</b> result surfaced to the user (Phase 18) rather than a silently wrong number — a wrong IRR is worse than none for an institutional investment decision                                                                                                                       |
| DEPLOY-01 | A rule-JSON publish (Phase 11) that passes the Human Verification Gate but contains a logic error (e.g. an inverted comparison) silently produces wrong FSI values for every subsequent evaluation in that jurisdiction | High     | Low        | Golden-dataset regression (Phase 27) includes at least one benchmark plot per published jurisdiction; a rule publish that changes a golden-dataset expected output requires explicit sign-off from a second reviewer, not just the authoring architect                                                   |

## Phase 33 — MVP (P0) Definition

The MVP is exactly the loop both Blueprint documents converge on independently, which is strong evidence it is the right cut:

Create Project → Draw Site (MapLibre + snapping) → Validate & Reproject Geometry (N1–N2)  
→ Add Constraints (roads, trees, easements) → Road-widening deduction (N3)  
→ Apply Regulation (Mumbai DCPR 2034 + Maharashtra UDCPR only, via the AST evaluator – N4)  
→ Straight-skeleton Setbacks (N5a) → Tree Buffer Subtraction (N6)  
→ Buildable Envelope + FSI/Height (N7) → 3D extrusion of the envelope (Three.js layer, footprint-only, no unit-level detail)  
→ Basic Financial Pro-Forma (GDV/TDC/NPV/IRR only – N11, high-level cost inputs, no debt-schedule detail)  
→ Save Immutable Version (Phase 26) → Generate PDF Report (N13)

**Explicitly excluded from MVP** (P1/P2/P3 per the Blueprint’s own prioritization, confirmed correct here): procedural unit packing/MILP, subterranean parking generator, NSGA-II optimization, Bengaluru/Delhi/other-jurisdiction rulesets beyond Mumbai+Maharashtra, regulation RAG/document ingestion, IFC/DXF export, real-time multi-user collaboration, cadastral vision-ingestion. **No mock calculations and no fake AI results anywhere in this loop** — every number in the P0 loop comes from a real deterministic worker call, including the financial pro-forma (using the real DCF formulas at a simplified input set, not placeholder arithmetic).

## Phase 34 — 90-Day Engineering Roadmap

Adopt the Blueprint’s week-by-week structure (Deliverable 16/\$36) as the sequencing baseline — it is well-ordered (infra → DB → auth → GIS canvas → geometry core → constraints → regulation engine → feasibility integration → layout/massing → financial → reporting → hardening) and matches dependency order correctly (e.g. the regulation engine (Week 7) is correctly sequenced after geometry core (Week 5) since rule evaluation needs `net_area / road_width` inputs the geometry engine produces). Adopted with one insertion: **Week 5.5 — Golden Dataset Seeding**, loading the 9 canonical benchmark plots into `/tests/golden` before Week 6’s constraint work begins, so every geometry PR from that point forward has a regression gate to run against rather than bolting testing on retroactively in Week 12.

| Weeks | Milestone                                                                                                       |
|-------|-----------------------------------------------------------------------------------------------------------------|
| 1     | Monorepo scaffold, TS/lint config, Docker Compose dev environment                                               |
| 2     | Postgres+PostGIS, Flyway migrations V001-V00N, GIST indexes, connection pooling                                 |
| 3     | OAuth2 auth, JWT+refresh-cookie issuance, protected routes, project CRUD                                        |
| 4     | MapLibre canvas in <code>useRef</code> isolation, India-scoped geocoding                                        |
| 5     | Geometry worker core: validation, UTM reprojection, Shoelace area, GEOS/Clipper2 boolean ops                    |
| 5.5   | Golden dataset (9 plots) seeded into <code>/tests/golden</code> ; CI gate wired                                 |
| 6     | CAD snapping/ortho-lock tools; road-widening deduction; tree buffer subtraction                                 |
| 7     | Rule JSON schema + AST compiler/evaluator (Phase 11); Mumbai DCPR 2034 encoded                                  |
| 8     | Feasibility DAG orchestrator (N1-N7) wired end-to-end; job-status API                                           |
| 9     | Core placement, corridor routing, initial NBC-compliant room decomposition                                      |
| 10    | Three.js massing layer wired into MapLibre; DCF engine (GDV/TDC/NPV/IRR)                                        |
| 11    | Headless-Chromium report renderer; React-PDF prospectus template                                                |
| 12    | Golden-dataset + property-fuzzing + financial-zero-variance gates all green; security scan; private beta deploy |

## Phase 35 — 6-12 Month Expansion Roadmap

Adopt the Blueprint’s phased expansion (Months 4-6: additional jurisdictions — Delhi MPD 2021, Hyderabad G.O. Ms. 168, Chennai TNCDBR 2019, Kolkata KMC Rules, Jharkhand JBBL — plus IFC/DXF export and Yjs-based real-time collaboration and Monte-Carlo financial sensitivity; Months 7-12: automated cadastral vision-ingestion, WebGPU solar/shadow simulation, and additional international jurisdictions) as correctly sequenced: it defers collaboration and vision-ingestion (both genuinely hard, high-uncertainty subsystems) past the jurisdictions/export work that has a clearer, lower-risk implementation path, which is the right risk ordering. One adjustment: **NSGA-II optimization and Bengaluru RMP ingestion**, listed as P1/P2 in the Blueprint’s MVP feature-priority table but not explicitly placed on the Months-4-6/7-12 timeline, are inserted at **Month 4** (immediately following MVP hardening) since they are P1/P2 features, not Month-7+ features, per the Blueprint’s own prioritization matrix — the two documents are slightly inconsistent with each other here (one gives a feature-priority table, the other a monthly roadmap, and they don’t fully line up); this plan resolves the inconsistency in favor of the more granular priority table.

## Phase 36 — Exact Implementation Order (single sequence, dependency-resolved)

- Repo scaffold + CI skeleton (Phase 4)
- Database schema + migrations + golden-dataset seed (Phase 6, 27)
- Auth + RBAC + tenant RLS (Phase 28)
- GeoJSON contract + shared schema package (Phase 7)
- Geometry worker: validation, reprojection, boolean ops (Phase 8, 9)
- Map canvas + CAD snapping (Phase 25) — *can run in parallel with 5 once the contract from step 4 is frozen*
- Constraint engine + road-widening/tree-buffer deduction (Phase 10)
- Regulation rule schema + AST compiler/evaluator + Mumbai/Maharashtra rule files (Phase 11)
- Feasibility DAG orchestrator, nodes N1-N7 (Phase 13)
- 3D massing (envelope-only) (Phase 17)
- Financial engine, DCF formulas, golden worked-example test (Phase 18)
- Report renderer (Phase 23’s export pool, PDF only)
- Versioning/audit-log immutability (Phase 26) — *retrofit onto every table introduced from step 2 onward once the pattern is proven on one table*
- Security hardening pass + observability wiring (Phase 28, 29)
- MVP beta.** Then, in priority order: layout/unit-packing MILP (14) → parking engine (15) → additional jurisdictions (regulation data files only, no new code) → NSGA-II optimization (20) → RAG/document-AI ingestion (12) → IFC/DXF export → real-time collaboration (CRDT) → cadastral vision-ingestion.

## Summary of Where This Plan Diverges From the Blueprint

- Regulation evaluator internals** (Phase 11) — the single largest addition; the Blueprint specifies the rule *data format* correctly but never the *evaluator* that consumes it. Without this, “declarative, version-controlled regulation engine” is a description, not a buildable system.

2. **Feasibility pipeline formalized as an executable DAG with idempotency/caching semantics** (Phase 13), rather than the Blueprint's prose/math sequence — needed to make “every calculation reproducible” operationally true.
3. **Kafka deferred to Enterprise horizon only**, resolving an internal inconsistency between the Blueprint's phased evolution strategy (which defers it) and its Tier-4 architecture diagram (which shows it at MVP).
4. **NSGA-II chromosome encoding and constraint-domination handling specified** (Phase 20) — the Blueprint names NSGA-II correctly but not its encoding, which is the actual implementation work.
5. **MILP formulation sketch for unit demising** (Phase 14) given explicitly, since the Blueprint names OR-Tools/MILP as the right tool but not the decision variables/objective/constraints.
6. **Database-level immutability enforcement** (Phase 26 trigger) added — the Blueprint defines an audit table but relies on application-level discipline to keep it append-only, which is not a reliable guarantee on its own.
7. **Three additional risk entries** (PERF-01, OPT-01, WORKER-01, UX-01, FIN-02, DEPLOY-01) covering failure modes implied by the architecture but not named in either Blueprint's risk register.
8. **Structural grid module's “conceptual, not licensed structural design” boundary** elevated from a documentation note to an enforced product/export requirement (Phase 16), given its liability significance.

Everything else in the two source Blueprint documents — the 32–38-operation geometry matrix, the multi-jurisdictional statutory tables, the full financial formula set, the security configuration, the cloud cost model, and the database DDL — was independently reviewed and found correct as specified, and is adopted rather than re-derived.